# Handy-App „Stromzähler SDM120" mit MIT App Inventor

Schritt-für-Schritt-Anleitung für eine Android-App. Die App verbindet sich per **Bluetooth Classic** mit dem D1 Mini ESP32 und zeigt die Messwerte des SDM120 live an.

---

## 0. Schnellstart mit der fertigen Projektdatei

Im Ordner `app/` liegt **`Stromzaehler.aia`**. Sie enthält das komplette Projekt aus den Kapiteln 4 und 5 (Designer und Blöcke):

1. In App Inventor *Projects → Import project (.aia) from my computer* wählen.
2. `Stromzaehler.aia` auswählen. Das Projekt öffnet sich mit fertigem Layout.
3. Im Blocks-Editor kontrollieren, ob alle 10 Blockgruppen da sind und keine roten Warnungen zeigen.
4. Die App als APK bauen oder per AI Companion auf dem Handy starten.

Gegenüber der Anleitung fehlt nur das optionale Leistungsdiagramm aus Kapitel 6. Die Bluetooth-Berechtigungen werden als Text angefragt (`android.permission.BLUETOOTH_CONNECT` / `…_SCAN`), damit die Datei auch mit älteren App-Inventor-Versionen läuft.

Meldet App Inventor beim Import einen Fehler, etwa zur Version des Projekts, notiere die genaue Meldung. Die Datei wird dann angepasst. Die Kapitel 4 und 5 beschreiben denselben Aufbau zum Nachbauen von Hand.

---

## 1. Funktionsumfang

- Liste der gekoppelten Bluetooth-Geräte, Verbinden und Trennen per Knopf
- Großanzeige von **Zählerstand (kWh)** und **aktueller Leistung (W)**
- Detailwerte: Spannung, Strom, Schein- und Blindleistung, Leistungsfaktor, Frequenz, Bezug, Einspeisung
- Statusanzeige: verbunden, Messung ok, Zähler antwortet nicht, keine Daten
- Knopf **Info**: Firmwareversion, IP-Adresse, WLAN-Signal, Fehlerzähler des ESP
- Optional: Liniendiagramm der Wirkleistung

---

## 2. Protokoll (Firmware → App)

Solange ein Handy verbunden ist, sendet der ESP32 **jede Sekunde** eine Textzeile mit Zeilenende `\n` (Byte 10):

```
$SDM;1;231.4;1.23;245.7;284.6;-143.2;0.863;50.01;1234.567;0.000;1234.567
```

| Position in der Liste (App Inventor zählt ab 1) | Inhalt | Einheit |
|---|---|---|
| 1 | Kennung `$SDM` | – |
| 2 | gültig: `1` = Messung ok, `0` = Zähler antwortet nicht | – |
| 3 | Spannung | V |
| 4 | Strom | A |
| 5 | Wirkleistung (negativ = Einspeisung) | W |
| 6 | Scheinleistung | VA |
| 7 | Blindleistung | var |
| 8 | Leistungsfaktor | – |
| 9 | Frequenz | Hz |
| 10 | Bezug (Import) | kWh |
| 11 | Einspeisung (Export) | kWh |
| 12 | Gesamt | kWh |

Fehlt ein Wert noch, zum Beispiel direkt nach dem Start, steht dort `-`.

Schickt die App den Befehl `I` mit Zeilenende, antwortet der ESP mit einer Infozeile:

```
$INF;2.0;192.168.1.57;-61;812;3;226;4071
```

| Position | Inhalt |
|---|---|
| 1 | Kennung `$INF` |
| 2 | Firmwareversion |
| 3 | IP-Adresse (`-` ohne WLAN) |
| 4 | WLAN-Signal RSSI in dBm |
| 5 | erfolgreiche Messzyklen |
| 6 | fehlgeschlagene Messzyklen |
| 7 | letzter Modbus-Fehlercode (dezimal, siehe README Kapitel 9) |
| 8 | Laufzeit in Sekunden |

## 3. Vorbereitung am Handy

1. Den ESP32 mit der Firmware einschalten.
2. Am Handy unter *Einstellungen → Bluetooth* nach neuen Geräten suchen und **„SDM120-Zaehler" koppeln**. Fragt das Handy nach einer PIN, nimm `1234` oder `0000`, meist ist keine nötig.
3. Die App zeigt später nur **gekoppelte** Geräte an.

---

## 4. Designer: Komponenten anlegen

Lege die Komponenten in genau dieser Reihenfolge an. Die Namen sind wichtig, weil die Blöcke in Kapitel 5 sie verwenden.

### 4.1 Screen1 (Eigenschaften)

| Eigenschaft | Wert |
|---|---|
| AppName | Stromzaehler |
| Title | Stromzähler SDM120 |
| AlignHorizontal | Center |
| BackgroundColor | Dark Gray (oder eigene Farbe, z. B. `#202020`) |
| ScreenOrientation | Portrait |
| Scrollable | an |

### 4.2 Sichtbare Komponenten

| Nr. | Komponente (Palette) | Name | Wichtige Eigenschaften |
|---|---|---|---|
| 1 | Layout → HorizontalArrangement | `haKnoepfe` | Width: Fill parent, AlignHorizontal: Center |
| 2 | User Interface → ListPicker (in `haKnoepfe`) | `lpVerbinden` | Text: `Verbinden`, Title: `Gerät wählen` |
| 3 | User Interface → Button (in `haKnoepfe`) | `btnTrennen` | Text: `Trennen`, Enabled: aus |
| 4 | User Interface → Button (in `haKnoepfe`) | `btnInfo` | Text: `Info`, Enabled: aus |
| 5 | User Interface → Label | `lblStatus` | Text: `nicht verbunden`, FontSize: 14, TextColor: Light Gray |
| 6 | User Interface → Label | `lblTitelZaehler` | Text: `Zählerstand`, FontSize: 16, TextColor: White |

| Nr. | Komponente (Palette) | Name | Wichtige Eigenschaften |
|---|---|---|---|
| 7 | User Interface → Label | `lblZaehlerstand` | Text: `– kWh`, FontSize: 40, FontBold: an, TextColor: White |
| 8 | User Interface → Label | `lblLeistung` | Text: `– W`, FontSize: 28, FontBold: an, TextColor: Yellow |
| 9 | Layout → TableArrangement | `taWerte` | Columns: 2, Rows: 8, Width: Fill parent |
| 10 | 8 × Label in **Spalte 1** von `taWerte` | `lblN1` … `lblN8` | Text siehe unten, TextColor: Light Gray, FontSize: 16 |
| 11 | 8 × Label in **Spalte 2** von `taWerte` | `lblSpannung`, `lblStrom`, `lblSchein`, `lblBlind`, `lblPF`, `lblFrequenz`, `lblImport`, `lblExport` | Text: `–`, TextColor: White, FontSize: 16, TextAlignment: right |
| 12 | User Interface → Label | `lblInfo` | Text: leer, FontSize: 12, TextColor: Light Gray |

Texte für die Namensspalte `lblN1` … `lblN8`: Spannung, Strom, Scheinleistung, Blindleistung, Leistungsfaktor, Frequenz, Bezug, Einspeisung.

### 4.3 Nicht sichtbare Komponenten

| Komponente (Palette) | Name | Eigenschaften |
|---|---|---|
| Connectivity → BluetoothClient | `BluetoothClient1` | **DelimiterByte: 10** (wichtig!), CharacterEncoding: UTF-8, Secure: an |
| Sensors → Clock | `Clock1` | **TimerInterval: 200**, TimerEnabled: **aus**, TimerAlwaysFires: aus |

**DelimiterByte = 10** sorgt dafür, dass `ReceiveText` mit der Länge `-1` genau bis zum Zeilenende liest. Steht hier der Standardwert 0, kommen unvollständige oder zusammengeklebte Zeilen an.

---

## 5. Blöcke

Die Blöcke sind hier als Text beschrieben. Eingerückte Zeilen stecken im Block darüber.

### 5.1 Globale Variablen

```
initialize global letzterEmpfang to 0
initialize global zeile to ""
initialize global teile to create empty list
```

### 5.2 Start: Bluetooth-Berechtigungen anfragen (Android 12 und neuer)

```
when Screen1.Initialize
  call Screen1.AskForPermission  permissionName: Permission BluetoothConnect
  call Screen1.AskForPermission  permissionName: Permission BluetoothScan
  set lblStatus.Text to "nicht verbunden"
```

Gibt es in deiner App-Inventor-Version im Permission-Dropdown kein *BluetoothConnect* oder *BluetoothScan*, steck stattdessen einen Textblock ein: `"android.permission.BLUETOOTH_CONNECT"` bzw. `"android.permission.BLUETOOTH_SCAN"`.

### 5.3 Geräteliste füllen

```
when lpVerbinden.BeforePicking
  set lpVerbinden.Elements to BluetoothClient1.AddressesAndNames
```

### 5.4 Verbinden

```
when lpVerbinden.AfterPicking
  if  call BluetoothClient1.Connect  address: lpVerbinden.Selection
  then
    set lblStatus.Text to "verbunden – warte auf Daten …"
    set lblStatus.TextColor to Green
    set global letzterEmpfang to Clock1.SystemTime
    set btnTrennen.Enabled to true
    set btnInfo.Enabled to true
    set lpVerbinden.Enabled to false
    set Clock1.TimerEnabled to true
  else
    set lblStatus.Text to "Verbindung fehlgeschlagen"
    set lblStatus.TextColor to Red
```

`lpVerbinden.Selection` enthält „MAC-Adresse + Name". `Connect` kommt damit zurecht.

### 5.5 Trennen (als Prozedur, damit sie auch bei Verbindungsverlust nutzbar ist)

```
to trennen
  set Clock1.TimerEnabled to false
  call BluetoothClient1.Disconnect
  set btnTrennen.Enabled to false
  set btnInfo.Enabled to false
  set lpVerbinden.Enabled to true
```

```
when btnTrennen.Click
  call trennen
  set lblStatus.Text to "getrennt"
  set lblStatus.TextColor to Light Gray
```

### 5.6 Info anfordern

```
when btnInfo.Click
  call BluetoothClient1.SendText  text: join("I", "\n")
```

Den Zeilenumbruch `\n` gibst du im Textblock direkt als `\n` ein.

### 5.7 Empfangen (Clock1, alle 200 ms)

```
when Clock1.Timer
  if  BluetoothClient1.IsConnected
  then
    if  call BluetoothClient1.BytesAvailableToReceive  >  0
    then
      set global zeile to  call BluetoothClient1.ReceiveText  numberOfBytes: -1
      call verarbeiteZeile  text: get global zeile
    if  (Clock1.SystemTime – get global letzterEmpfang)  >  5000
    then
      set lblStatus.Text to "keine Daten vom ESP"
      set lblStatus.TextColor to Orange
  else
    call trennen
    set lblStatus.Text to "Verbindung verloren"
    set lblStatus.TextColor to Red
```

Ein Timer-Durchlauf liest **eine** Zeile. Der ESP sendet nur jede Sekunde eine, und der Timer läuft alle 200 ms. Dadurch entsteht nie ein Rückstau.

### 5.8 Zeile auswerten

```
to verarbeiteZeile  text
  set global teile to  split  text: trim(get text)  at: ";"

  if  (length of list get global teile) = 12
      and  (select list item  list: get global teile  index: 1) = "$SDM"
  then
    set global letzterEmpfang to Clock1.SystemTime

    if  (select list item  index: 2) = "1"
    then  set lblStatus.Text to "● Messung ok"         ; TextColor Green
```

```
    else  set lblStatus.Text to "● Zähler antwortet nicht" ; TextColor Red

    set lblZaehlerstand.Text to  join( item 12, " kWh" )
    set lblLeistung.Text     to  join( item 5,  " W" )
    set lblSpannung.Text     to  join( item 3,  " V" )
    set lblStrom.Text        to  join( item 4,  " A" )
    set lblSchein.Text       to  join( item 6,  " VA" )
    set lblBlind.Text        to  join( item 7,  " var" )
    set lblPF.Text           to       item 8
    set lblFrequenz.Text     to  join( item 9,  " Hz" )
    set lblImport.Text       to  join( item 10, " kWh" )
    set lblExport.Text       to  join( item 11, " kWh" )

  else if  (length of list get global teile) = 8
           and  (select list item  index: 1) = "$INF"
  then
    set global letzterEmpfang to Clock1.SystemTime
    set lblInfo.Text to  join( "FW ", item 2,
                               " · IP ", item 3,
                               " · WLAN ", item 4, " dBm",
                               " · OK ", item 5,
                               " / Fehler ", item 6,
                               " · Code ", item 7 )
```

`item n` ist hier eine Abkürzung für `select list item list: get global teile index: n`.

**Optional: Einspeisung farbig anzeigen.** Ergänze nach dem Setzen von `lblLeistung`:

```
if  is number?( item 5 )  and  item 5 < 0
then  set lblLeistung.TextColor to Green     ; Einspeisung
else  set lblLeistung.TextColor to Yellow    ; Bezug
```

---

## 6. Optional: Leistungsverlauf als Diagramm

1. Im Designer unter *Charts* ein **Chart** einfügen. Name `Chart1`, Type: `line`, Height: 200 px, Width: Fill parent.
2. In das Chart ein **ChartData2D** ziehen. Name `cdLeistung`, Label: `Wirkleistung W`.
3. Neue globale Variable anlegen: `initialize global x to 0`.
4. Am Ende des `$SDM`-Zweigs in `verarbeiteZeile` ergänzen:

```
if  is number?( item 5 )
then
  set global x to  get global x + 1
  call cdLeistung.AddEntry  x: get global x   y: item 5
  if  get global x  ≥  120
  then
```

```
    call cdLeistung.Clear
    set global x to 0
```

Das zeigt die letzten rund 2 Minuten bei einem Wert pro Sekunde und beginnt dann von vorn.

---

## 7. Test und Fehlersuche

| Symptom | Ursache / Lösung |
|---|---|
| Liste im ListPicker ist leer | ESP32 nicht gekoppelt (Kapitel 3) oder Bluetooth-Berechtigung abgelehnt. Berechtigung in den App-Einstellungen nachträglich erlauben. |
| `Connect` schlägt fehl | ESP32 aus, oder ein anderes Handy bzw. ein PC-Terminal ist schon verbunden. Bluetooth SPP erlaubt nur **eine** Verbindung. |
| „verbunden – warte auf Daten" bleibt stehen | **DelimiterByte nicht 10**, oder `BT_ENABLED` in `config.h` ist 0. |
| Werte zeigen `-` und Status „Zähler antwortet nicht" | Bluetooth funktioniert, aber der Modbus-Teil hat ein Problem. Siehe README Kapitel 9, der Info-Knopf zeigt den Fehlercode. |
| Werte springen oder Zeilen sind abgeschnitten | Der Timer liest zu früh. Prüfe `ReceiveText` mit `-1` und DelimiterByte 10. |
| Test im **Emulator** | Der Emulator hat **kein Bluetooth**. Teste mit einem echten Handy über den AI Companion (WLAN) oder als installierte APK. |

**Tipp zum ersten Test ohne App:** Mit einer Terminal-App wie „Serial Bluetooth Terminal" lässt sich prüfen, ob die `$SDM`-Zeilen ankommen. Danach die Terminal-Verbindung trennen, sonst kann sich die App nicht verbinden.

---

## 8. Erweiterungsideen

- **Schalten vom Handy aus:** In der Firmware weitere Befehle in `btBefehlAusfuehren()` ergänzen, zum Beispiel `R1` / `R0` für ein Relais. In der App sendest du sie per `SendText`.
- **Tagesverbrauch:** Beim Verbinden den Zählerstand in TinyDB speichern, wenn das Datum gewechselt hat. Danach die Differenz zum aktuellen Stand anzeigen.
- **Schwellwert-Alarm:** Liegt die Wirkleistung über einem einstellbaren Wert, die Hintergrundfarbe ändern oder `Notifier.ShowAlert` aufrufen.